

TECHNISCHE UNIVERSITÄT BERLIN

Exposé

Real-Time Speech Recognition for Streaming Transcription: A Causal Adaptation of Whisper Models

Proposal by Karl Wenzel

Master Thesis supervised by

Prof. Dr.-Ing. Dorothea Kolossa and M.Sc. Serdar Gareayaghi

May 28, 2026

1 Abstract

Whisper is a state-of-the-art speech recognition model capable of transcribing speech with very high accuracy and a low real-time factor. Although originally developed for the transcription of pre-recorded audio, its potential for streaming applications has been increasingly investigated. Over the past two years, several implementations have attempted to utilize or adapt Whisper for streaming applications.

Most recently, CarelessWhisper, a fully streaming architecture, achieved considerably lower latency by utilizing a causal encoder and caching of encoder keys and values (KV's). Therefore, the aim of this thesis is to follow a similar approach, adapting and fine-tuning various Whisper models. This involves finetuning the model using low-rank adaption as well as architecture changes. Furthermore, the model will be fine-tuned on datasets containing natural speech and German language. These changes will be evaluated on various criteria such as WER, RTF and latency.

2 Introduction

Speech is the most natural form of human communication [14]. As such, many applications benefit from speech-based interaction. Automatic speech recognition (ASR), the process of transcribing speech to text, is used in accessible learning [18], dictation systems [14], voice interaction with mobile devices [14], live captioning [13] and many other domains. The ASR pipeline usually involves several steps such as feature extraction, an acoustic model, a language model and a decoder [14].

Automatic speech recognition research dates back to the 1950s, when Bell Labs’ digit recognizer demonstrated the ability to identify isolated spoken digits [1]. Since the 1980s, hidden markov models (HMMs) were used to model speech as a stochastic process [4]. These HMM-based models remained state-of-the-art until the 2010s, when deep neural networks (DNNs) brought the next major leap in accuracy and robustness. Lee et. al. [7] applied convolutional neural networks (CNNs) to audio data to replace the acoustic model in the speech recognition process, while later Hannun et. al. [3] introduced end-to-end speech recognition using recurrent neural networks (RNNs). With the advent of transformer architectures, first introduced by Vaswani et al. [17], they were soon applied to speech recognition, for example by Dong et al. [2]. Unlike recurrent models, transformer-based architectures enable better parallelization during training. One of the most prominent transformer-based ASR systems to date is Whisper [11], illustrating the potential of transformer based ASR models and motivating the investigation of real-time speech recognition approaches presented in this work.

3 Related Work and Theoretical Background

3.1 Whisper

Whisper, introduced by Radford et. al. [11], achieves a word error rate (WER) as low as 2.7% on the English LibriSpeech test-clean benchmark using the large-v2 model [6]. The authors of Whisper attribute this performance gain primarily to scaling the training dataset to over 680,000 hours of audio (of which 563,000 hours are English, 117,000 hours are multilingual), approximately two orders of magnitude more than previous models. Whisper explicitly disregards the traditional standard of carefully curated datasets and does not employ self-supervision or self-training [11].

As Whisper is based on the transformer architecture (unlike previously used RNNs, where this information is implicitly encoded by the sequential nature of the algorithm), sinusoidal positional encodings are used to encode the positions of input tokens [17]. Using the formulas:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

where pos is the position and i is the dimension and d is the model dimension, multiple dimensions of sinusoidal encodings are summed with the input embeddings. Thereby, the model can make use of the positions of the input embeddings while still supporting parallelization. Still, these embeddings, later described as *absolute* position embeddings [16], have some limitations that will be discussed in the Methodology section of this thesis.

The model was trained on 30-second audio segments, enabling it to capture longer-range contextual dependencies and thereby improve transcription accuracy. However, this extended context window also makes the default Whisper model less suitable for real-time speech recognition. The challenge in real-time ASR is to achieve both high transcription accuracy and low latency, typically measured by the real-time factor (RTF). Processing smaller chunks as they arrive - rather than complete audio files as intended for offline

use - prevents full utilization of the model’s context window, which can increase the word error rate. As Whisper is designed and trained to operate on 30-second audio segments, any shorter input will be zero-padded to be able to process it. Consequently, attention must still be computed across the full 30 seconds, introducing computational overhead. Therefore, modifications to the Whisper architecture are required to make it suitable for real-time ASR.

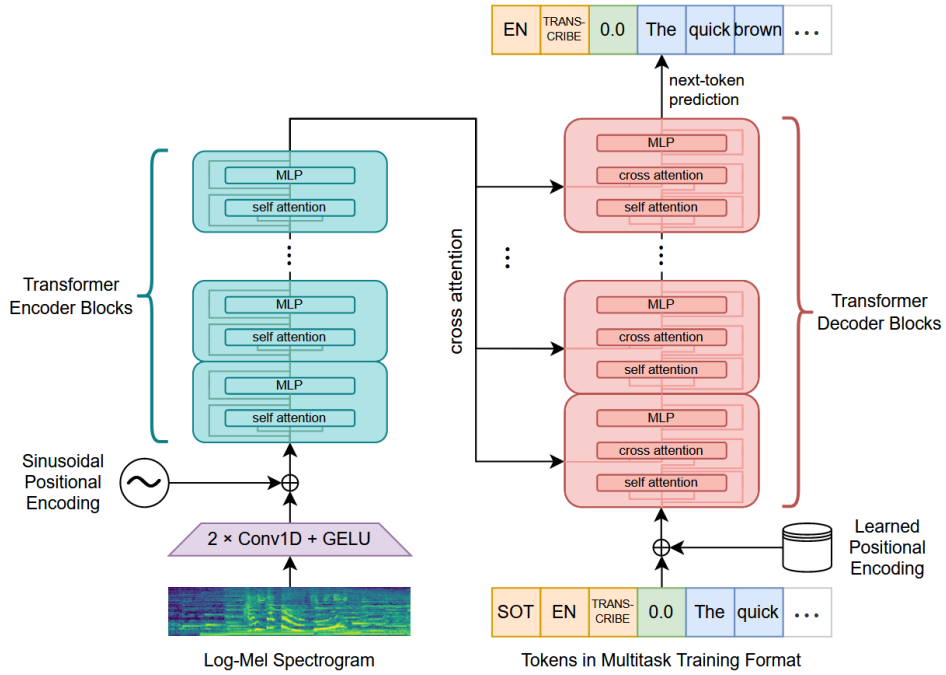


Figure 1: The Whisper architecture [11]

3.2 Realtime automatic speech recognition models

Multiple studies have proposed modifications to Whisper to enable real-time ASR. The following section outlines key approaches, namely UFAL-Whisper [8], U2-Whisper [19], and CarelessWhisper [6], and discusses their respective architecture changes, fine-tuning methods and performance metrics.

Whisper-Streaming (UFAL-Whisper) [8] employs an audio buffer to concatenate received audio samples. In a set interval, the audio buffer, start-

ing at the last sentence-end, is processed by Whisper. A local agreement policy is then used to compare the last two updates, and the longest common prefix is appended to the output. It achieves an WER of between 2.78% (with 1.5 seconds chunk size) and 10.71% WER (0.1 seconds chunk size) on the LibriSpeech test-clean dataset and 5.30% to 16.80% on test-other when using Whisper model size medium, as reported by Zhou et. al. [19]. Due to the local agreement policy, UFAL-Whisper produces finalized output only when two consecutive predictions align, resulting in an effective finalization latency of at least twice the chunk size. Consequently, UFAL-Whisper is significantly less suitable for actual real-time use.

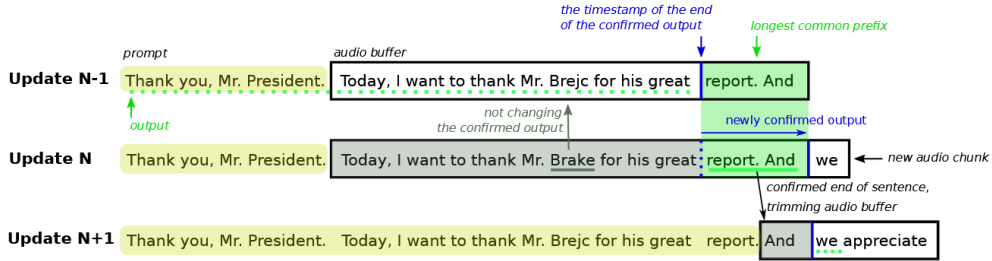


Figure 2: Illustration of the local agreement policy of UFAL-Whisper, processing three consecutive updates [8]. In this case, the longest common prefix “report. And” is appended to the output.

U2-Whisper [19] uses fine-tuning and architecture changes to turn whisper into a realtime ASR model. The model is trained using causal attention masks, allowing efficient key and value caching during inference. An additional Connectionist Temporal Classification (CTC) decoder is added. Using beam-search, the CTC-decoder produces the top-k streaming transcripts, which are subsequently rescored using the original Whisper-decoder. The best scoring partial transcript is chosen and appended to the output. Both decoders use input from the original Whisper encoder.

The CTC-decoder is trained on only a subset of the token space of the original Whisper-decoder, restricting it to the first 8000 tokens. The Top-K CTC-outputs are retokenized in parallel for the Whisper-decoder. This hybrid tokenizer assures sufficient generalization of the CTC-decoder on chal-

lenging test sets, despite limited fine-tuning.

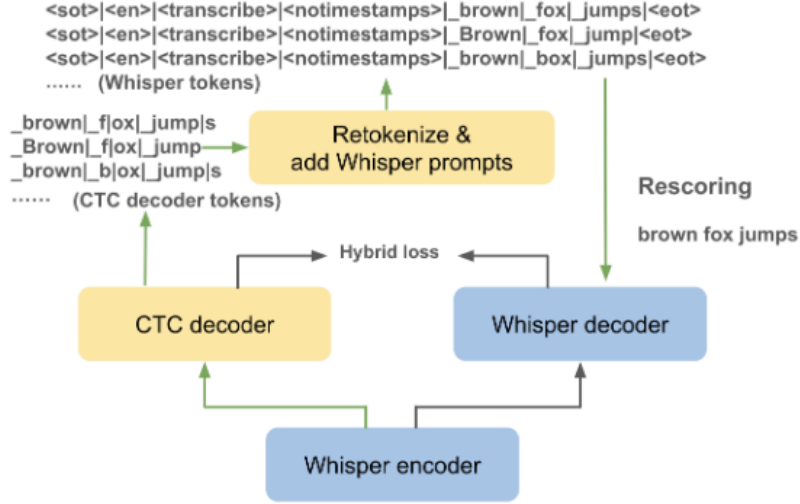


Figure 3: Illustration of the U2-Whisper architecture [19]. An additional CTC-decoder is added to the original Whisper architecture.

The main advantage of U2-Whisper over UFAL-Whisper is the ability to perform well on smaller chunk sizes, where it outperforms UFAL-Whisper. It achieves an WER of between 2.71% (with 1.5 seconds chunk size) and 5.43% WER (0.1 seconds chunk size) on the LibriSpeech test-clean dataset and 7.31% to 13.27% on test-other when using the Whisper model size medium [19]. It is also more computationally efficient, although a big downside is that output is only emitted at endpoint detection, usually during pauses, rendering the smaller chunk sizes less relevant for the end user. Consequently, while U2-Whisper achieves good WER metrics, it may not fit the requirements for real-time deployment.

CarelessWhisper is another approach to real-time ASR [6]. It adapts Whisper to a fully streaming architecture using fine-tuning with causal attention masks and encoder KV caching. To improve training efficiency, the authors employ low-rank adaptation (LoRA). Fine-tuning is performed on the weakly aligned LibriSpeech training set, which is force-aligned using the Montreal Forced Aligner (MFA) [9].

Krichli et. al. attend to the performance problem of realtime ASR, that mainly stems from repeatedly encoding the whole input buffer every time it is processed. They provide a mathematical proof that, due to the causal mask of the encoder, the information contained in encoded frames does not change when a new frame is received, enabling CarelessWhisper to utilize key and value caching (KV-caching) of the encoded frames. This caching mechanism eliminates much of the encoding overhead and largely accounts for the strong computational performance of CarelessWhisper.

It manages to outperform the other streaming solutions greatly in terms of WER on lower chunk sizes. The authors reported 5.29% WER with 0.3 seconds chunk size when using Whisper model size large-v2, while UFAL-Whisper performs with a 7.50% WER on the same configuration. When testing with a chunk sizes of 0.04 seconds, the difference becomes even larger with a 7.76% WER of CarelessWhisper and 78.40% of UFAL-Whisper [6]. CarelessWhisper also has much better computational performance, with around 0.1 seconds average latency compared to a 0.46 seconds latency of UFAL-Whisper on 0.3 seconds chunk size using the large-v2 model size [6]. Consequently, the real-time factor (RTF) of UFAL-Whisper is higher than 1, while a RTF of 1 or lower usually means a model can be considered real-time. With such a high RTF, UFAL-Whisper can not be used for real-time purposes, while the metrics for CarelessWhisper are promising. Using smaller model sizes (like base or tiny), which perform comparably in terms of WER, it may also be the best contender to run on standalone devices.

One limitation of CarelessWhisper is a cache clear, that is executed every 30 seconds. It stems from the 30 seconds context length of the Whisper architecture and the absolute sinusoidal positional embeddings used in the encoder. After clearing the cache, inference continues with only a few milliseconds of re-encoded context (from the previous audio buffer) and the current chunk. This procedure may lead to latency spikes (due to a 600ms start buffer) and a higher WER when streaming for more than 30 seconds, especially after every cache clear, when the encoder context is reset fully. While CarelessWhisper employs encoder self-attention and cross-attention KV-caching, Krichli et. al. observe, that applying KV-caching to decoder self-attention leads to significant degradation in WER. They attribute this

to the fact, that decoder self-attention depends on representations that already incorporate cross-attention outputs; because the cross-attention is non-causal, the decoder self-attention becomes unstable over newly arriving chunks.

Model	Model Size	WER (test-clean)	WER (test-other)
UFAL-Whisper	base	21.43	29.43
CarelessWhisper	base	10.63	21.59
UFAL-Whisper	large-v2	27.64	35.27
CarelessWhisper	large-v2	6.33	13.06

Table 1: Reported WER on LibriSpeech datasets with 0.1s chunk size [6].

3.3 Efficient model training and inference

Low-rank adaption (LoRA) is an effective technique to accelerate the fine-tuning process of large models such as Whisper as described by Song et. al. [15]. The weights of the original model are frozen and pairs of rank decomposition matrices are trained instead, greatly reducing the number of trainable parameters. A feed forward layer defined as

$$f_i(x) = W_i x + b_i$$

where $W_i \in \mathbb{R}^{d_1 \times d_2}$ and $b_i \in \mathbb{R}^{d_1}$ are the weights and biases respectively, can be extended to

$$f_i(x) = (W_i + \Delta W_i)x + b_i, \quad \Delta W_i = B_i A_i$$

where weights W_i and biases b_i are frozen and $B_i \in \mathbb{R}^{d_1 \times r}$ and $A_i \in \mathbb{R}^{r \times d_2}$ are the trainable matrices. r is the adjustable rank parameter, which is chosen to be much smaller than d_1 and d_2 to achieve the goal of reduced trainable parameter count.

Curriculum Fine-Tuning of LoRA Layers: Curriculum learning is a method in which the training process of neural networks is structured similarly to the human learning process: Starting with simpler tasks and gradually progressing to more complex ones [5]. Usually, Whisper is trained with

a chunk size of 30 seconds. When fine-tuning for a specific chunk size, the context is shrunk “at once”. Performing fine-tuning on relatively large chunk sizes first and then progressing to smaller chunk sizes may help to model to adapt better, leading to faster convergence.

Song et al. [15] describe a LoRA warm start approach in the context of multilingual ASR. When training a new language-specific LoRA, they initialize its weights with those of the most similar language. This process can be adapted for curriculum learning, where instead of warm-starting from the most similar language, the best checkpoint from the next larger chunk size is used.

3.4 Beam Search Decoding

The beam search algorithm is a widely used method for generating the most likely sequence of tokens from a probabilistic model, such as an ASR system. In contrast to greedy decoding, which selects the most probable token at each step, beam search maintains a fixed number of candidate sequences (the "beam width") and explores multiple paths simultaneously. This allows it to find better overall sequences by considering alternative paths and not being limited by a locally optimal choice. At each decoding step, the algorithm expands all current beams by generating possible next tokens and their probabilities. Then, the top-k sequences are selected based on their cumulative probabilities, where k is the beam width. This process continues until a stopping criterion is met, such as reaching a maximum sequence length or generating an end-of-sequence token.

3.4.1 Beam Search Stability Check

While beam search, unlike greedy decoding, is usually more robust due to multiple explored paths, in a streaming scenario, it may fail to decode the tokens at the end of the current audio prefix. This is due to insufficient context for a correct prediction or even the audio buffer ending mid-word. A stability check can be integrated into the beam search decoding process, as proposed by Krichli et. al. [6]. This stability check reevaluates the last n tokens of all current beams. If they are not still in the top-k most probable decoded tokens for that beam and position, considering the current audio context, they are regenerated. This approach allows the decoder to correct potential errors in the last few tokens and recover from wrong predictions caused by chunk boundaries. A look back distance n is used to determine how many of the last tokens to reevaluate.

3.4.2 Stopping Criterion

Usually, beam search decoding continues decoding, until a sufficient number of hypotheses have generated an end-of-sequence token. However, Krichli et. al. [6] argue, that this can lead to hallucinations and repeated phrases in the

streaming ASR setting. Therefore, they propose to stop decoding for each timestep, once any of the beams generates an end-of-sequence token.

To enable german language training, a german language normalizer was added. We used the `german_transliterate` normalizer [\[link\]](#), which is a lightweight normalizer, that handles common issues such as abbreviations, currency symbols, and other features that exist in the written form and can not be directly recognized by a speech recognition model. By applying this normalizer to the training data, we can ensure that the text is in a consistent format, which can improve the performance of the speech recognition model when processing German language input. The normalizer is also integrated into the evaluation pipeline, ensuring that the metrics are calculated on the normalized text.

4 Methodology

4.1 Experiments

4.1.1 Fine-Tuning

A first set of experiments involves fine-tuning Whisper models using the exact same approach outlined by Krichli et. al. [6]. First, fine-tuning on the Whisper-base model will be performed using a force-aligned LibriSpeech test dataset. This configuration exactly replicates one from the CarelessWhisper paper and serves as a benchmark and starting point for future experiments. Then, the same process will be repeated for the original Whisper models Large-v3 and Turbo, that have not been used and evaluated yet by Krichli et. al.

When fine-tuning on the Multilingual LibriSpeech (MLS) dataset, Krichli et. al. reported poor results for several multilingual transcription tasks [6]. This might be due to insufficient training data, leading to poor generalization. Therefore, fine-tuning on a more extensive dataset of German language might provide better results.

As the LibriSpeech as well as the MLS datasets only include audiobook narration, the models presumably perform worse on natural language. Employing a natural language dataset might produce a model capable of performing better in scenarios resembling actual usecases like live captioning.

Finetuning will be performed for different chunksizes: 40ms, 100ms, 300ms, 600ms and 1s. Curriculum learning described in the related work section will be used to potentially accelerate the finetuning process, using the best checkpoint of a bigger chunksizes to initialize the LoRA weights for the next smaller chunksize.

4.1.2 Architecture Changes

As mentioned above, when transcribing for more than 30 seconds, Careless-Whisper performs a cache clear and thereby practically restarts. Presumably, encoded input frames could not be moved to a different location in the 30-second context due to fixed sinusoidal positional encodings. To create a true

streaming model without this constraint, it will be investigated if there is an approach to change this. The possibility of a sliding cache will be investigated using architecture changes and fine-tuning. Possible approaches to enable a sliding cache are rotary positional encodings (RoPE), introduced by Su et. al. [16], or adding biases to query-key attention scores, penalizing them according to distance (ALiBi), introduced by Press et. al. [10]. Another approach could be to reencode the last few seconds of the 30-seconds context.

CarelessWhisper also employs a 600-ms start buffer, that adds latency when first starting the model, repeated for every cash clear. Krichli et. al. do not provide explanation why the start buffer was implemented, although one could theorize that it improved the models metrics by avoiding initial hallucinations due to insufficient context. It will be investigated, if this is the case or if the buffer can be removed. Additional fine-tuning may be required.

CarelessWhisper does not use decoder KV-caching, due to the cross-attention attending to future encoded frames. This non-causal cross-attention leads to decoder self-attention becoming unstable over new chunks. In the future research section, the CarelessWhisper authors propose to use a causal mask for the cross-attention in the decoder during finetuning. This would prevent the decoder from attending to future encoder outputs at any given inference step, enforcing causal cross-attention. The authors hypothesize that, with causal cross-attention, KV caching of decoder self-attention could be employed without significant degradation in WER, while improving RTF and latency.

4.2 Datasets

4.2.1 Forced Alignment

Krichli et. al. use the weakly aligned LibriSpeech dataset for fine-tuning [6], force-aligned with the Montreal Forced Aligner (MFA) [9]. In the experiments, the HMM-GNN based MFA will be used as a benchmark but the NeMo Forced Aligner (NFA) [12], which is a more recent approach based on the Conformer model, will also be employed. It is expected that the NFA will provide more accurate token boundaries as it relies on learned neural

context, while also running faster as it can be parallelized on the GPU.

4.2.2 Datasets

Several datasets will be used for fine-tuning. LibriSpeech contains English audiobook narration. The LRS and LRW datasets contain natural speech. The selected common voice (CV) dataset contains scripted (non-natural) German language.

Dataset	Avg. len	Shortest	Longest	Samples	Hours
LibriSpeech	11.06	1.06	35.16	39,665	122
LRS2_v1	2.16	0.52	6.16	23,017	14
LRW	1.16	1.16	1.16	538,766	174
LRS3_pretrain	12.37	0.76	668.48	118,516	407
LRS3_test	2.32	0.56	6.20	1,452	1
LRS3_trainval	3.39	0.48	6.20	31,982	30
CV Scr 23.0 Ger	5.29			1,010,623	1484

Table 2: Dataset statistics. Lengths in seconds, hours rounded.

4.3 Evaluation

All produced models will be evaluated on several criteria to assess their suitability for the usecase of streaming ASR. In addition to the word error rate, several metrics related to the realtime nature of the task will be used.

Word Error Rate (WER). The standard WER measures the proportion of incorrectly recognized words in the final transcription and is defined as:

$$\text{WER} = \frac{S + D + I}{N}$$

where S , D , and I denote the number of substitutions, deletions, and insertions and N is the total number of words in the reference transcription.

Relative Word Error Rate (RWER). RWER, introduced by Krichli et al. [6], evaluates the accuracy of partial transcriptions over time in streaming ASR and is defined as:

$$\text{RWER}(y, \hat{y}) = \frac{\sum_{\rho} [I(y_{\rho}, \hat{y}_{\rho}) + D(y_{\rho}, \hat{y}_{\rho}) + S(y_{\rho}, \hat{y}_{\rho})]}{\sum_{\rho} [C(y_{\rho}, \hat{y}_{\rho}) + D(y_{\rho}, \hat{y}_{\rho}) + S(y_{\rho}, \hat{y}_{\rho})]}$$

where ρ denotes the relative prefix index, $\hat{y}_{\rho}$ is the hypothesis prefix produced by the model at relative step ρ , and y_{ρ} is the corresponding ground-truth prefix. I , D , S and C denote insertions, deletions, substitutions and correct predictions respectively. RWER measures how accurately the model transcribes partial audio segments without penalizing temporal misalignment.

The time index τ used in the original paper has been replaced with ρ to avoid confusion with the time index used in ARWER.

Aligned Relative Word Error Rate (ARWER). ARWER [6] extends RWER by incorporating temporal alignment between the hypothesis and the ground-truth words expected to have been transcribed by each real-time point:

$$\text{ARWER}(y, \hat{y}) = \frac{\sum_{\tau} [I(y_{\tau}, \hat{y}_{\tau}) + D(y_{\tau}, \hat{y}_{\tau}) + S(y_{\tau}, \hat{y}_{\tau})]}{\sum_{\tau} [C(y_{\tau}, \hat{y}_{\tau}) + D(y_{\tau}, \hat{y}_{\tau}) + S(y_{\tau}, \hat{y}_{\tau})]}$$

where τ denotes the real-time index. Unlike RWER, ARWER penalizes delayed emissions, causing models with higher latency to exhibit worse ARWER.

Real-Time Factor (RTF). RTF quantifies how efficiently the model processes audio in real time:

$$\text{RTF} = \frac{T_{\text{process}}}{T_{\text{audio}}}$$

where T_{process} is the total processing time and T_{audio} is the duration of the input audio. Values below 1 indicate faster-than-real-time performance. The RTF is an important measure of the computational efficiency of the model.

Latency. Latency measures the delay between receiving an input audio segment and producing the corresponding output:

$$\text{Latency} = T_{\text{output}} - T_{\text{input}}$$

With latency, the noticable lag for the end user can be directly measured. Lower latency is critical for real-time ASR usecases.

RTF and Latency are device-specific and can therefore vary based on the hardware used for inference.

Strict WER (SWER). During testing, it was observed, that the model sometimes changes transcribed tokens several seconds after they were first emitted. This is presumably due to additional context being available, leading to the model correcting its previous hypothesis. While this may aid the WER, it is undesirable for real-time applications, as the user would see the changed transcriptions and possibly perceive the output as unstable. A simple measure to force stable transcriptions is to freeze transcribed tokens once they are emitted and only allow changing the last k tokens. To measure how the correctness of model output degrades over smaller k values, a strict WER can be calculated. The SWER is defined as:

$$\text{SWER}_k = \frac{S_k + D_k + I_k}{N}$$

where k controls the allowed revision window. Smaller values of k impose stricter stability constraints. $k = 0$ corresponds to fully immutable streaming output and $k = \infty$ corresponds to the standard WER. By evaluating SWER for different values of k , the stability of the model’s transcriptions can be assessed, providing insights into the trade-off between accuracy and stability in real-time ASR applications.

Word instability rate (WIR). To further quantify the stability of the model’s transcriptions, the Word Instability Rate (WIR) can be calculated. WIR measures the proportion of words that are revised after their initial emission, with multiple revisions of the same word counting multiple times. It is defined as:

$$\text{WIR}_k = \frac{R_k}{N}$$

where R is the total number of word revisions, N is the total number of words in the reference transcription and k describes the suffix length of a partial transcription, where word changes are allowed and do not contribute to the total revisions. Increasing k allows for more revisions without counting towards instability, while smaller values of k impose stricter stability requirements.

4.4 Replicability Study

In order to verify that our training setup matches the one used by Krichli et. al. [6], we conducted a replicability study, reproducing the training of the model on the same dataset, with the same hyper-parameters and comparing the results.

We produced a LibriSpeech-960 dataset by merging the original train-clean-100, train-clean-360 and train-other-500 subsets. This results in a dataset of 960 hours of English audiobook narration, which is the same dataset used by Krichli et. al. [6] for training their models. This dataset was force-aligned using the Montreal Forced Aligner [9] to obtain word-level timestamps. An external evaluation script was used to compute the WER, RWER, ARWER and other metrics on the test-clean and test-other subsets of LibriSpeech. We evaluate both our trained models and the models provided by Krichli et. al. and compare the results.

4.5 True streaming ASR with Alibi Embeddings and Sliding Cache

4.5.1 The Problem of Sinusoidal Positional Encodings

The realtime Whisper implementation of Krichli et. al. [6] has a significant constraint: After 30 seconds of audio, the model performs a full context clear. Only a few frames of audio from the previous buffer are kept by re-encoding them, but the rest of the context is lost. This leads to a significant temporary drop in accuracy after 30 seconds, as the model can no longer leverage the full context of the audio. It also does not allow for a truly streaming ASR system, as the model cannot maintain a continuous understanding of the audio beyond 30 seconds. A naive solution to this problem would be to just slide the cached encoded frames by pruning the chunk 1, so the old chunk 2 becomes the new chunk 1, and appending the newly arriving chunk to the end of the context. But this hits a fundamental problem of the Whisper architecture, which currently uses fixed sinusoidal positional encodings. These encodings are based on the absolute position of the input frames and are applied before calculating encoder attention, which irreversibly adds them to the encoded frames. If frames were moved to a different location in the context, their positional encodings would no longer be correct. This leads to a significant drop in accuracy, as Whisper was trained on these positional encodings and relies on them to understand the temporal structure of the audio. For a sliding context with sinusoidal positional encodings, the whole retained context would need to be re-encoded with new positional encodings, which would be computationally expensive and renders the whole concept of an encoder cache absurd.

4.5.2 Alibi Embeddings

There are several solutions to make positional embeddings more flexible, mostly aimed at extending the context length of transformer models to longer ones than those encountered during training. One such approach are rotary positional encodings (RoPE), introduced by Su et. al. [16]. Rotary positional encodings are based on the concept of rotating attention queries and keys, to

encode positional information. Presumably, when sliding the context, once-calculated queries and keys could just be rotated further, without needing to calculate them again. Another approach is adding biases to query-key attention scores, penalizing them according to distance (ALiBi), introduced by Press et. al. [10]. These biases are applied after the queries and keys are calculated, also enabling a sliding context. For ease of implementation, we chose to implement ALiBi embeddings, as they only require adding a bias tensor to the attention scores, which can be accomplished in 'just a few lines of code' [10].

4.5.3 LoRA-Finetuning with Alibi Embeddings

LoRA fine-tuning works under the assumption, that conceptual changes to the models weights, to adapt to a new domain or task, are low-dimensional. We show, that the model can adapt to the new ALiBi embeddings with a low-dimensional LoRA fine-tuning, without needing to change the original weights of the model. To go even further, the model could adapt to the realtime streaming task as well as to the new positional embeddings with a single LoRA fine-tuning, reaching a similar accuracy as the baseline model that was trained with sinusoidal positional encodings.

positional embedding method	WER
sinusoidal (baseline)	8,47%
ALiBi	9,95%

Table 3: WER as a function of the positional embedding type. Measured on LibriSpeech-test-clean for a Whisper-base model fine-tuned for 300ms chunk-size RT-transcription on the LibriSpeech-960 dataset.

4.5.4 Sliding Cache

With the ALiBi embeddings, a sliding cache can be implemented by pruning the first chunk of the context and appending the newly arriving chunk to the end of the context. The cache saves the query and key tensors before applying the ALiBi bias, so they can be reused without needing to be recalculated.

This setup allows for a growing cache for the first 30 seconds of streaming and a sliding cache after that, which is always approximately 30 seconds in length.

4.6 Freezing hypothesis prefixes sometimes improves accuracy

While measuring strict WER (SWER) with different values of k , it was observed, that the SWER can actually be better than the standard WER for some values of k .

strict_k	STRICT_WER
0	17.85%
1	9.84%
2	7.88%
3	7.84%
4	7.80%
5	7.82%
7	7.90%
10	7.99%
15	8.25%
20	8.47%
Base WER	8.47%

Table 4: Strict WER as a function of the allowed revision window k . Measured on the test-clean subset of LibriSpeech for a Whisper-base model fine-tuned for 300ms chunk-size RT-transcription.

References

- [1] K. H. Davis, R. Biddulph, and S. Balashek. “Automatic Recognition of Spoken Digits”. In: *The Journal of the Acoustical Society of America* 24.6 (Nov. 1, 1952), pp. 637–642. ISSN: 0001-4966, 1520-8524. DOI: [10.1121/1.1906946](https://pubs.aip.org/jasa/article/24/6/637/618458/Automatic-Recognition-of-Spoken-Digits). URL: <https://pubs.aip.org/jasa/article/24/6/637/618458/Automatic-Recognition-of-Spoken-Digits> (visited on 11/01/2025).
- [2] Linhao Dong, Shuang Xu, and Bo Xu. “Speech-Transformer: A No-Recurrence Sequence-to-Sequence Model for Speech Recognition”. In: *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). ISSN: 2379-190X. Apr. 2018, pp. 5884–5888. DOI: [10.1109/ICASSP.2018.8462506](https://ieeexplore.ieee.org/document/8462506). URL: <https://ieeexplore.ieee.org/document/8462506> (visited on 10/07/2025).
- [3] Awni Hannun et al. *Deep Speech: Scaling up end-to-end speech recognition*. Dec. 19, 2014. DOI: [10.48550/arXiv.1412.5567](https://arxiv.org/abs/1412.5567). arXiv: [1412.5567\[cs\]](https://arxiv.org/abs/1412.5567). URL: <http://arxiv.org/abs/1412.5567> (visited on 10/07/2025).
- [4] B H Juang and Lawrence R Rabiner. “Automatic Speech Recognition – A Brief History of the Technology Development”. In: ().
- [5] Jisu Kim and Juhwan Lee. *Strategic Data Ordering: Enhancing Large Language Model Performance through Curriculum Learning*. May 13, 2024. DOI: [10.48550/arXiv.2405.07490](https://arxiv.org/abs/2405.07490). arXiv: [2405.07490\[cs\]](https://arxiv.org/abs/2405.07490). URL: <http://arxiv.org/abs/2405.07490> (visited on 11/05/2025).
- [6] Tomer Krichli, Bhiksha Raj, and Joseph Keshet. *Careless Whisper: Turning Whisper into a Causal Streaming Model*. Aug. 17, 2025. DOI: [10.48550/arXiv.2508.12301](https://arxiv.org/abs/2508.12301). arXiv: [2508.12301\[cs\]](https://arxiv.org/abs/2508.12301). URL: <http://arxiv.org/abs/2508.12301> (visited on 09/26/2025).
- [7] Honglak Lee et al. “Unsupervised feature learning for audio classification using convolutional deep belief networks”. In: ().

- [8] Dominik Macháček, Raj Dabre, and Ondřej Bojar. *Turning Whisper into Real-Time Transcription System*. Sept. 21, 2023. DOI: [10.48550/arXiv.2307.14743](https://doi.org/10.48550/arXiv.2307.14743). arXiv: [2307.14743\[cs\]](https://arxiv.org/abs/2307.14743). URL: <http://arxiv.org/abs/2307.14743> (visited on 10/13/2025).
- [9] Michael McAuliffe et al. “Montreal Forced Aligner: Trainable Text-Speech Alignment Using Kaldi”. In: *Interspeech 2017*. Interspeech 2017. ISCA, Aug. 20, 2017, pp. 498–502. DOI: [10.21437/Interspeech.2017-1386](https://doi.org/10.21437/Interspeech.2017-1386). URL: https://www.isca-archive.org/interspeech_2017/mcauliffe17_interspeech.html (visited on 10/13/2025).
- [10] Ofir Press, Noah A. Smith, and Mike Lewis. *Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation*. Apr. 22, 2022. DOI: [10.48550/arXiv.2108.12409](https://doi.org/10.48550/arXiv.2108.12409). arXiv: [2108.12409\[cs\]](https://arxiv.org/abs/2108.12409). URL: <http://arxiv.org/abs/2108.12409> (visited on 11/05/2025).
- [11] Alec Radford et al. *Robust Speech Recognition via Large-Scale Weak Supervision*. Dec. 6, 2022. DOI: [10.48550/arXiv.2212.04356](https://doi.org/10.48550/arXiv.2212.04356). arXiv: [2212.04356\[eess\]](https://arxiv.org/abs/2212.04356). URL: <http://arxiv.org/abs/2212.04356> (visited on 09/13/2025).
- [12] Elena Rastorgueva, Vitaly Lavrukhin, and Boris Ginsburg. “NeMo Forced Aligner and its application to word alignment for subtitle generation”. In: ().
- [13] Pablo Romero-Fresco and Nazaret Fresno. “The accuracy of automatic and human live captions in English”. In: *Linguistica Antverpiensia, New Series – Themes in Translation Studies* 22 (Dec. 13, 2023). ISSN: 2295-5739. DOI: [10.52034/lans-tts.v22i.774](https://doi.org/10.52034/lans-tts.v22i.774). URL: <https://artojs02.uantwerpen.be/index.php/LANS-TTS/article/view/774> (visited on 10/06/2025).
- [14] Karpagavalli S and Chandra E. “A Review on Automatic Speech Recognition Architecture and Approaches”. In: *International Journal of Signal Processing, Image Processing and Pattern Recognition* 9.4 (Apr. 30, 2016), pp. 393–404. ISSN: 20054254, 20054254. DOI: [10.14257/ijssip.2016.9.4.34](https://doi.org/10.14257/ijssip.2016.9.4.34). URL: http://article.nadiapub.com/IJSIP/vol9_no4/34.pdf (visited on 09/13/2025).

- [15] Zheshu Song et al. *LoRA-Whisper: Parameter-Efficient and Extensible Multilingual ASR*. June 7, 2024. DOI: [10.48550/arXiv.2406.06619](https://doi.org/10.48550/arXiv.2406.06619). arXiv: [2406.06619\[eess\]](https://arxiv.org/abs/2406.06619). URL: <http://arxiv.org/abs/2406.06619> (visited on 10/21/2025).
- [16] Jianlin Su et al. “RoFormer: Enhanced transformer with Rotary Position Embedding”. In: *Neurocomputing* 568 (Feb. 1, 2024), p. 127063. ISSN: 0925-2312. DOI: [10.1016/j.neucom.2023.127063](https://doi.org/10.1016/j.neucom.2023.127063). URL: <https://www.sciencedirect.com/science/article/pii/S0925231223011864> (visited on 11/05/2025).
- [17] Ashish Vaswani et al. *Attention Is All You Need*. Aug. 2, 2023. DOI: [10.48550/arXiv.1706.03762](https://doi.org/10.48550/arXiv.1706.03762). arXiv: [1706.03762\[cs\]](https://arxiv.org/abs/1706.03762). URL: <http://arxiv.org/abs/1706.03762> (visited on 10/07/2025).
- [18] Mike Wald and Keith Bain. “Universal access to communication and learning: the role of automatic speech recognition”. In: *Universal Access in the Information Society* 6.4 (Feb. 1, 2008), pp. 435–447. ISSN: 1615-5297. DOI: [10.1007/s10209-007-0093-9](https://doi.org/10.1007/s10209-007-0093-9). URL: <https://doi.org/10.1007/s10209-007-0093-9> (visited on 10/06/2025).
- [19] Haoran Zhou et al. *Adapting Whisper for Streaming Speech Recognition via Two-Pass Decoding*. version: 1. June 13, 2025. DOI: [10.48550/arXiv.2506.12154](https://doi.org/10.48550/arXiv.2506.12154). arXiv: [2506.12154\[cs\]](https://arxiv.org/abs/2506.12154). URL: <http://arxiv.org/abs/2506.12154> (visited on 10/13/2025).